CocoConnect

Technology Comparison Document

Group 38 · Prepared by: S.N. Vidanage (2023/IS/108)

1. Purpose

This document compares the main technology options for each layer of CocoConnect and explains which one was selected and why.

2. What the technology must support

The requirements analysis showed the system needs to:

- Run on Android and iOS phones, used by people with limited digital experience.
- Support English, Sinhala and Tamil.
- Handle structured, related data: users, estates, listings, orders, negotiations and ratings.
- Find buyers, estates and service providers by location and distance.
- Keep passwords secure and give each role access only to its own functions.
- Be built by a student team within two semesters, at little or no cost.

3. Mobile frontend

Criteria	Flutter ✓	React Native	Native (Kotlin + Swift)
Platforms	Android and iOS (and web) from one codebase	Android and iOS from one codebase	Separate app for each platform
Language	Dart	JavaScript / TypeScript	Kotlin and Swift
UI consistency	Draws its own widgets, so screens look the same on every phone	Uses native components, so look can differ between platforms	Fully native look
Performance	Compiled to native code; smooth	Good; a bridge to native can slow complex screens	Best
Development speed	Fast; hot reload	Fast; hot reload	Slow; two codebases
Multilingual support	Built-in localisation	Through libraries	Built-in on each platform

Decision: **Flutter** – one codebase for Android and iOS, consistent screens for every user, and fast development for a student team.

4. Backend

Criteria	Node.js + Express.js ✓	Python + Django	Java + Spring Boot
Language	JavaScript	Python	Java
Learning curve	Low	Medium	High
REST API development	Lightweight and quick to build	Good, with Django REST Framework	Powerful, but more setup and code
Handling many requests	Non-blocking; handles many requests well	Good	Very good
Packages	Very large (npm)	Large (pip)	Large (Maven)
Fit for this project	Simple REST API for a mobile app	Better suited to admin-heavy web apps	Better suited to large enterprise systems

Decision: **Node.js with Express.js** – lightweight, quick to build REST APIs with, easy for the team to learn, and has packages for everything needed (database access, password hashing, tokens, API documentation).

5. Database

Criteria	PostgreSQL ✓	MySQL	MongoDB	Firebase Firestore
Type	Relational (SQL)	Relational (SQL)	Document (NoSQL)	Document (NoSQL), cloud
Related data	Strong: foreign keys, joins, transactions	Strong	Weak: no joins between collections	Weak
Location queries	Excellent with PostGIS	Basic	Good (geo indexes)	Limited
Cost	Free, open source	Free, open source	Free tier	Free tier, then pay per read
Hosting	Supabase managed hosting with a free tier	Many hosts	MongoDB Atlas	Google only

Decision: PostgreSQL, hosted on Supabase – the data is strongly related (a negotiation links a buyer, a listing and an order), transactions keep deals consistent, PostGIS supports location queries, and Supabase hosts it for free.

6. Authentication and security

Criteria	JWT (JSON Web Tokens) ✓	Server sessions	Firebase Authentication
How it works	Signed token stored on the phone and sent with each request	Session ID stored on the server	Google-hosted login service
Fit for mobile apps	Very good; no cookies needed	Weaker; built around browser cookies	Very good
Role-based access	Role kept inside the token and checked on every request	Possible	Needs extra setup
Dependence	None; runs in our own backend	None	Tied to Google

Decision: JWT, with bcrypt for password hashing – suits a mobile app talking to a REST API, carries the user's role for access checks, and keeps authentication in our own backend. Passwords are stored only as bcrypt hashes.

7. Maps and location

Criteria	Google Maps API	OpenStreetMap (flutter_map) ✓
Cost	Needs a billing account and API key; charged after free usage	Free; no API key
Map quality in Sri Lanka	Very good	Good
Place search	Google Places (paid)	Nominatim (free)
Flutter support	Official plugin	flutter_map package

Decision: OpenStreetMap with flutter_map – free with no billing account, good enough coverage for estates and towns, and free place search through Nominatim. Google Maps was the first option considered, but its billing requirement was a barrier for the project.

8. Version control and tools

Criteria	GitHub ✓	GitLab	Bitbucket
Team collaboration	Branches and pull requests	Branches and merge requests	Branches and pull requests
CI/CD	GitHub Actions	GitLab CI	Bitbucket Pipelines
Team familiarity	High	Low	Low
Cost	Free for students	Free tier	Free for small teams

Decision: GitHub – the whole team already uses it, it is free, and each member can work on their own branch.

Other tools: Swagger for API documentation, Postman for testing APIs, Figma for wireframes, and Android Studio with the Android emulator for running the app.

9. Summary of selected technologies

Layer	Selected technology	Main reason
Mobile app	Flutter (Dart)	One codebase for Android and iOS; consistent UI; fast development
Backend	Node.js + Express.js	Lightweight REST API; easy to learn; large package ecosystem
Database	PostgreSQL on Supabase	Related data with transactions; location queries; free hosting
Security	JWT + bcrypt	Token login suited to mobile; role-based access; hashed passwords
Maps	OpenStreetMap (flutter_map) + Nominatim	Free with no API key or billing
Version control	GitHub	Team familiarity; branches and pull requests; free